Линейная регрессия. Часть 2

В первой части мы рассмотрели теорию линейной регрессии. Теперь перейдем к практике.

Откроем блокнот к этому занятию

Сквозной пример

Данные и постановка задачи

Обратимся к хорошо знакомому нам датасету недвижимости в Бостоне.

```
boston = pd.read_csv('/content/boston.csv')
```

При этом нам нужно будет решить две основные задачи:

Задача 1. Научиться оценивать качество модели не только с точки зрения метрики, но и исходя из рассмотренных ранее допущений

модели. Эту задачу мы решим в три этапа.

- *Этап 1*. Построим базовую (baseline) модель линейной регрессии с помощью класса LinearRegression библиотеки sklearn и оценим, насколько выполняются рассмотренные выше допущения.
- *Этап 2*. Попробуем изменить данные таким образом, чтобы модель в большей степени соответствовала этим критериям.
- *Этап 3*. Обучим еще одну модель и посмотрим, как изменится результат.

Задача 2. С нуля построить модель множественной линейной регрессии и сравнить прогноз с результатом, полученным при решении первой задачи. При этом обучение модели мы реализуем двумя способами, а именно, через:

- метод наименьших квадратов; и
- метод градиентного спуска.

Разделение выборки

Мы уже не раз говорили про важность разделения выборки на обучающую и тестовую части. Сегодня же, с учетом того, что нам предстоит изучить много нового материала, мы опустим этот этап и будем обучать и тестировать модель на одних и тех же данных.

Исследовательский анализ данных

Теперь давайте более внимательно посмотрим на имеющиеся у нас данные. Как вы вероятно заметили, все признаки в этом датасете количественные, за исключением переменной CHAS.

```
boston.info()
```

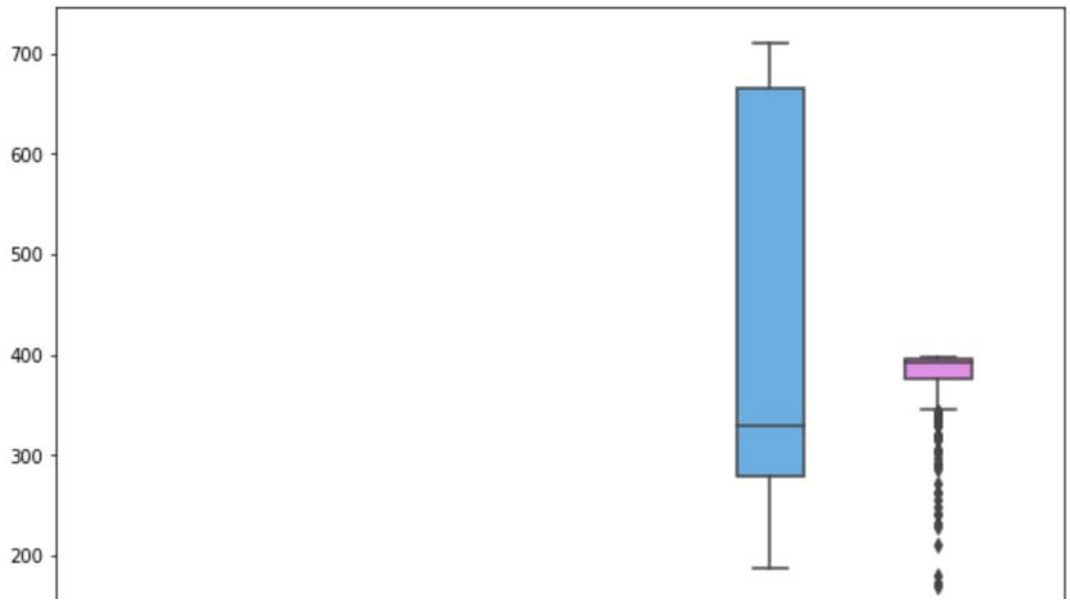
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
```

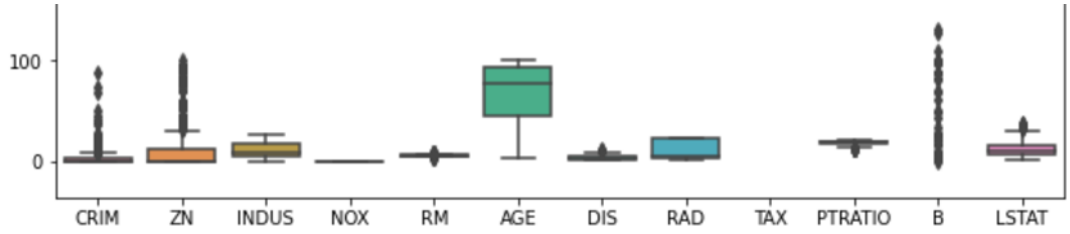
```
# мы видим, что переменная CHAS категориальная
boston.CHAS.value_counts()
```

```
0.0    471
1.0     35
Name: CHAS, dtype: int64
```

Посмотрим на распределение признаков с помощью boxplots.

```
sns.boxplot(data = boston.drop(columns = ['CHAS', 'MEDV']));
```

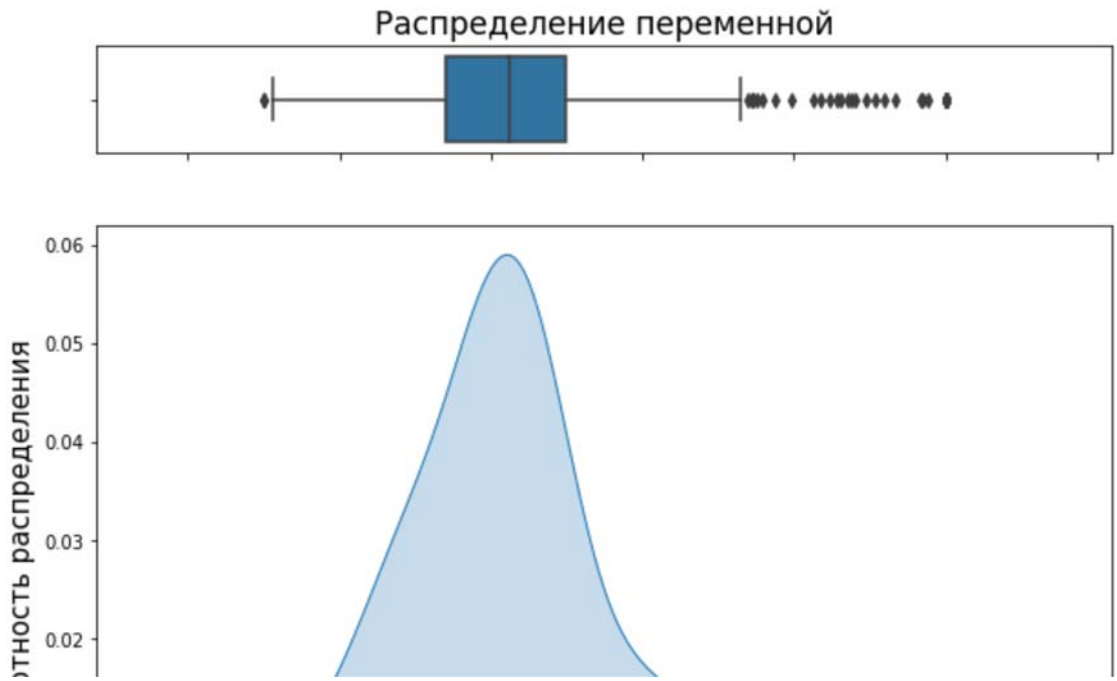


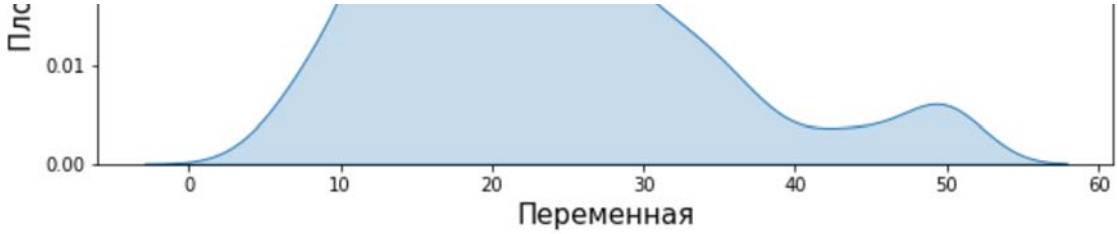


Посмотрим на распределение целевой переменной.

```
def box_density(x):
  # создадим два подграфика
  f, (ax_box, ax_kde) = plt.subplots(nrows = 2, # из двух строк
                                     ncols = 1, # и одного сто
                                     sharex = True, # оставим
                                     gridspec_kw = {'height_ra

  # в первом подграфике построим boxplot
  sns.boxplot(x = x, ax = ax_box)
  ax_box.set(xlabel = None)
  # во втором - график плотности распределения
  sns.kdeplot(x, fill = True)

  # зададим заголовок и подписи к осям
  ax_box.set_title('Распределение переменной', fontsize = 15)
  ax_kde.set_xlabel('Переменная', fontsize = 13)
  ax_kde.set_ylabel('Плотность распределения', fontsize = 13)

  plt.show()
```

```
box_density(boston.iloc[:, -1])
```





Посмотрим на корреляцию количественных признаков с целевой переменной.

```
boston.drop(columns = 'CHAS').corr().MEDV.to_frame().style.back
```

	MEDV
CRIM	-0.388305
ZN	0.360445
INDUS	-0.483725
NOX	-0.427321
RM	0.695360
AGE	-0.376955
DIS	0.249929
RAD	-0.381626
TAX	-0.468536
PTRATIO	-0.507787
B	0.333461
LSTAT	-0.737663
MEDV	1.000000

Используем точечно-бисериальную корреляцию для оценки взамосвязи переменной CHAS и целевой переменной.

```
def pbc(continuous, binary):

  # преобразуем количественную переменную в массив Numpy
  continuous_values = np.array(continuous)
  # классы качественной переменной превратим в нули и единицы
  binary_values = np.unique(binary, return_inverse = True)[1]

```

```
    # создадим две подгруппы количественных наблюдений
    # в зависимости от класса дихотомической переменной
    group0 = continuous_values[np.argwhere(binary_values == 0).fl
    group1 = continuous_values[np.argwhere(binary_values == 1).fl

    # найдем средние групп,
    mean0, mean1 = np.mean(group0), np.mean(group1)
    # а также длины групп и всего датасета
    n0, n1, n = len(group0), len(group1), len(continuous_values)

    # рассчитаем СКО количественной переменной
    std = continuous_values.std()

    # подставим значения в формулу
    return (mean1 - mean0) / std * np.sqrt( (n1 * n0) / (n * (n-1
```

```
pbc(boston.MEDV, boston.CHAS)
```

```
0.17543361629972762
```

Обработка данных

Пропущенные значения

Посмотрим, есть ли пропущенные значения.

```
boston.isnull().sum()
```

```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```

Выбросы

Удалим выбросы.

```
from sklearn.ensemble import IsolationForest

clf = IsolationForest(max_samples = 100, random_state = 42)
clf.fit(boston)

boston['anomaly'] = clf.predict(boston)
boston = boston[boston.anomaly == 1]
boston = boston.drop(columns = 'anomaly')
boston.shape
```

```
(402, 14)
```

При удалении выбросов важно помнить, что полное отсутствие вариантивности в данных не позволит выявить взаимосвязи.

Масштабирование признаков

Приведем признаки к одному масштабу (целевую переменную трогать не будем).

```
boston.iloc[:, :-1] = (boston.iloc[:, :-1] - boston.iloc[:, :-1
```

Замечу, что метод наименьших квадратов не требует масштабирования признаков, градиентному спуску же напротив необходимо, чтобы все значения находились в одном диапазоне (подробнее в дополнительных материалах).

Кодирование категориальных переменных

После стандартизации переменная CHAS также сохранила только два значения.

```
boston.CHAS.value_counts()
```

```
-0.182581    389
 5.463391     13
Name: CHAS, dtype: int64
```

Ее можно не трогать.

Построение модели

Создадим первую пробную (baseline) модель с помощью библиотеки sklearn.

baseline-модель

```
X = boston.drop('MEDV', axis = 1)
y = boston['MEDV']

from sklearn.linear_model import LinearRegression

model = LinearRegression()
y_pred = model.fit(X, y).predict(X)
```

Оценка качества

Диагностика модели, метрики качества и функции потерь

Вероятно, вы заметили, что мы использовали MSE и для обучения модели, и для оценки ее качества. Возникает вопрос, есть ли различие между функцией потерь и метрикой качества модели.

Функция потерь и метрика качества могут совпадать, а могут и не совпадать. Важно понимать, что у них разное назначение:

- функция потерь — это часть алгоритма; нам важно, чтобы эта

функция была дифференцируема (у нее была производная);

- производная метрики качества нас не интересует; метрика качества должна быть адекватна решаемой задаче.

MSE, RMSE, MAE, MAPE

MSE и RMSE

Для оценки качества RMSE предпочтительнее, чем MSE, потому что показывает, насколько ошибается модель в тех же единицах измерения, что и целевая переменная. Например, если диапазон целевой переменной от 80 до 100, а RMSE 20, то в среднем модель ошибается на 20-25 процентов.

Для того чтобы попрактиковаться, напишем собственную функцию.

```
# параметр squared = True возвращает MSE
# параметр squared = False возвращает RMSE
def mse(y, y_pred, squared = True):
  mse = ((y - y_pred) ** 2).sum() / len(y)

  if squared == True:
    return mse
  else:
    return np.sqrt(mse)
```

```
mse(y, y_pred), mse(y, y_pred, squared = False)
```

```
(9.980044349414225, 3.159120819059351)
```

Сравним с sklearn.

```
from sklearn.metrics import root_mean_squared_error

root_mean_squared_error(y, y_pred)
```

```
3.159120819059351
```

MAE

Приведем формулу.

$$MAE = \frac{\sum |y - \hat{y}|}{n}$$

Средняя абсолютная ошибка представляет собой среднее арифметическое абсолютной ошибки $\varepsilon = |y - \hat{y}|$ и использует те же единицы измерения, что и целевая переменная.

```
def mae(y, y_pred):
  return np.abs(y - y_pred).sum() / len(y)
```

```
mae(y, y_pred)
```

```
2.3556517952334177
```

```
from sklearn.metrics import mean_absolute_error

mean_absolute_error(y, y_pred)
```

```
2.3556517952334177
```

MAE часто используется при оценке качества моделей временных рядов.

MAPE

Средняя абсолютная ошибка в процентах (mean absolute percentage error) по сути выражает MAE в процентах, а не в абсолютных величинах, представляя отклонение как долю от истинных ответов.

$$MAPE = \frac{1}{n} \sum |\frac{y - \hat{y}}{y}|$$

Это позволяет сравнивать модели с разными единицами измерения между собой.

```
def mape(y, y_pred):
  return 1/len(y) * np.abs((y - y_pred) / y).sum()
```

```
mape(y, y_pred)
```

```
0.11819866589612749
```

```
from sklearn.metrics import mean_absolute_percentage_error

mean_absolute_percentage_error(y, y_pred)
```

```
0.11819866589612749
```

Коэффициент детерминации

В рамках вводного курса в ответах на вопросы к занятию по регрессии мы подробно рассмотрели коэффициент детерминации, R^2, его связь с RMSE, а также вопрос о том, зачем нужен скорректированный R^2. Как мы знаем, если использовать, например, класс LinearRegression, то эта метрика содержится в **методе .score()**.

```
model.score(X, y)
```

```
0.7965234359550825
```

Также можно использовать **функцию r2_score()** модуля metrics.

```
from sklearn.metrics import r2_score

r2_score(y, y_pred)
```

```
0.7965234359550825
```

Для скорректированного R^2 напишем собственную функцию.

```
def r_squared(x, y, y_pred):
  r2 = 1 - ((y - y_pred)** 2).sum()/((y - y.mean()) ** 2).sum()

  n, k = x.shape
  r2_adj = 1 - ((y - y_pred)** 2).sum()/((y - y.mean()) ** 2).su

  return r2, r2_adj
```

```
r_squared(X, y, y_pred)
```

```
(0.7965234359550825, 0.7965234359550825)
```

Диагностика модели

Теперь проведем диагностику модели в соответствии со сделанными ранее допущениями.

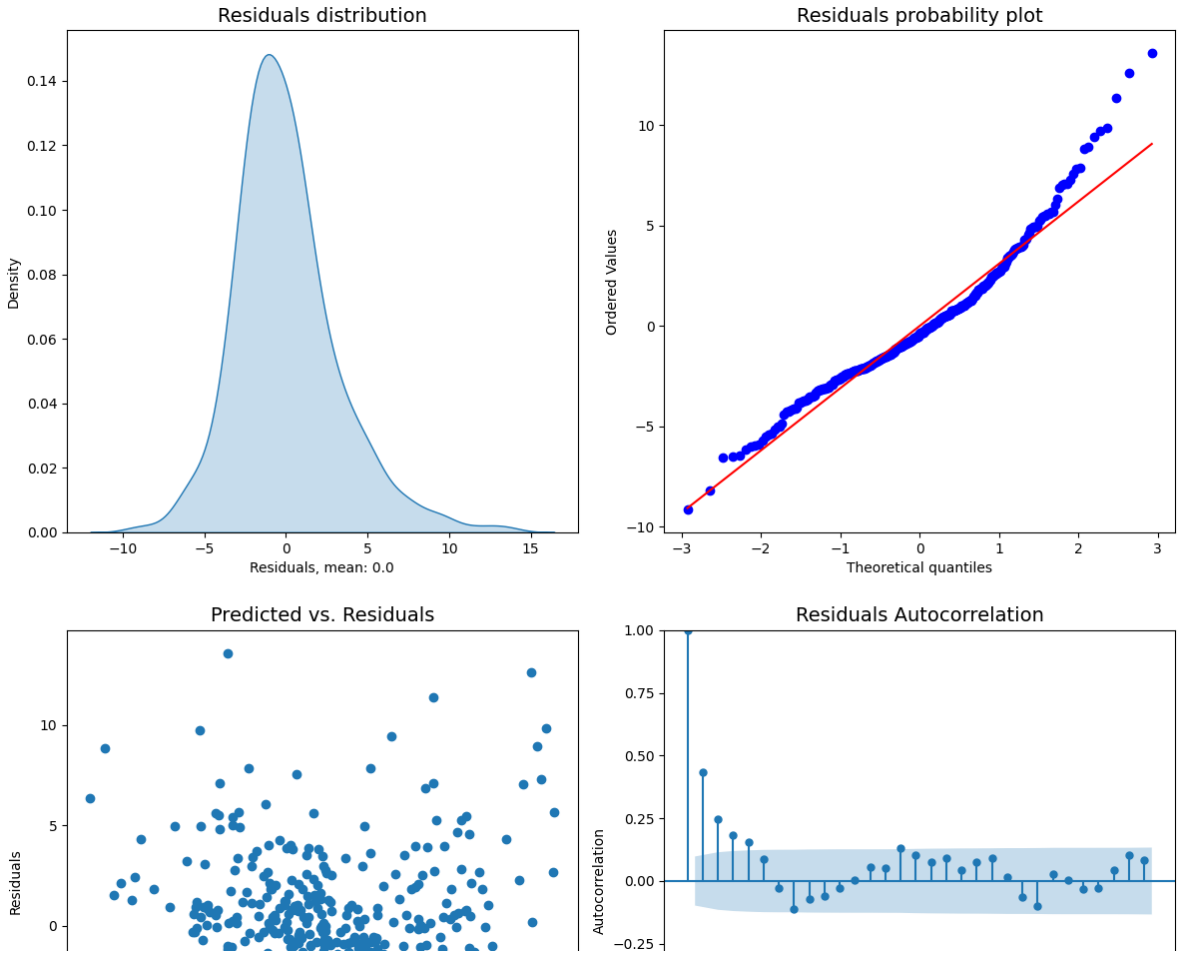
Анализ остатков и прогнозных значений

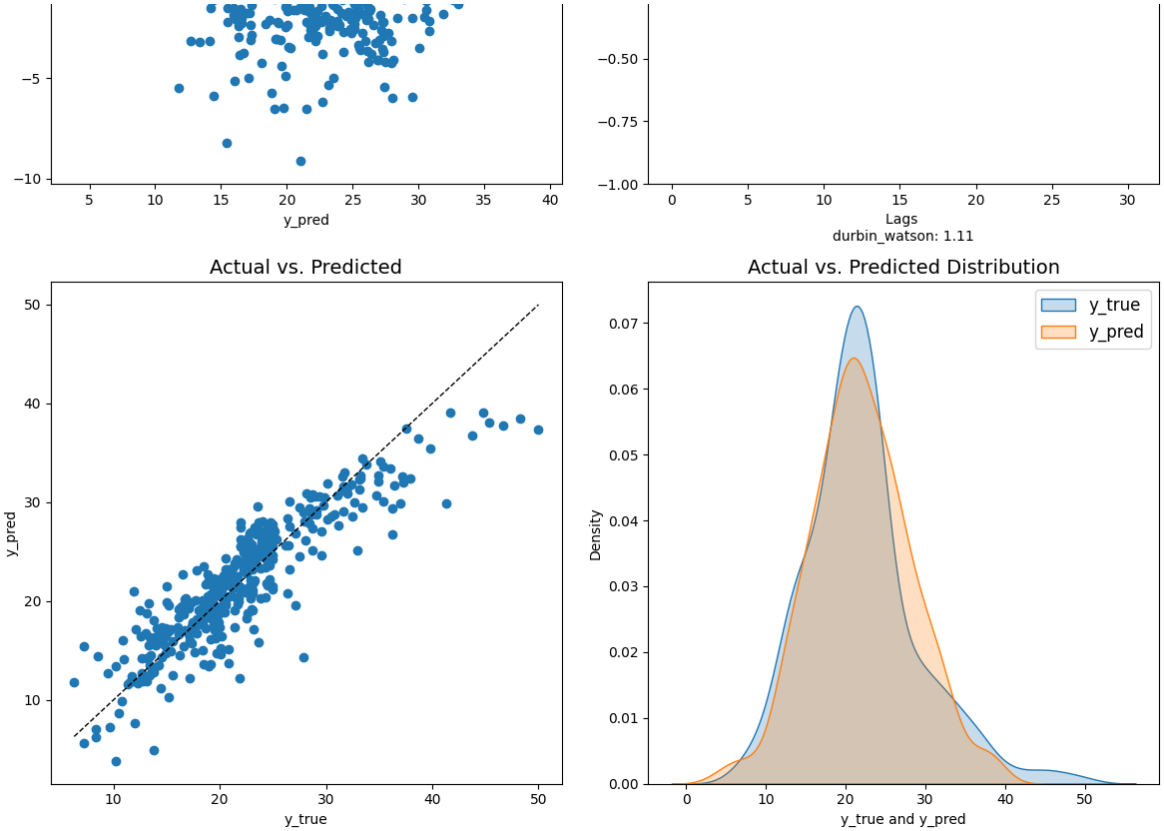
Напишем диагностическую функцию, которая сразу выведет несколько интересующих нас графиков и метрик, касающихся остатков и прогнозных значений.

```
from scipy.stats import probplot
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.stattools import durbin_watson

def diagnostics(y, y_pred):
  residuals = y - y_pred
  residuals_mean = np.round(np.mean(y - y_pred), 3)

  f, ((ax_rkde, ax_prob), (ax_ry, ax_auto), (ax_yy, ax_ykde)) =



  # в первом подграфике построим график плотности остатков
  sns.kdeplot(residuals, fill = True, ax = ax_rkde)
  ax_rkde.set_title('Residuals distribution', fontsize = 14)
  ax_rkde.set(xlabel = f'Residuals, mean: {residuals_mean}')
  ax_rkde.set(ylabel = 'Density')

  # во втором - график нормальной вероятности остатков
  probplot(residuals, dist = 'norm', plot = ax_prob)
  ax_prob.set_title('Residuals probability plot', fontsize = 14

  # в третьем - график остатков относительно прогноза
  ax_ry.scatter(y_pred, residuals)
  ax_ry.set_title('Predicted vs. Residuals', fontsize = 14)
  ax_ry.set(xlabel = 'y_pred')
  ax_ry.set(ylabel = 'Residuals')

  # в четвертом - автокорреляцию остатков
  plot_acf(residuals, lags = 30, ax = ax_auto)
```

```
    ax_auto.set_title('Residuals Autocorrelation', fontsize = 14)
    ax_auto.set(xlabel = f'Lags \ndurbin_watson: {durbin_watson(r
    ax_auto.set(ylabel = 'Autocorrelation')

    # в пятом - сравним прогнозные и фактические значения
    ax_yy.scatter(y, y_pred)
    ax_yy.plot([y.min(), y.max()], [y.min(), y.max()], "k--", lw
    ax_yy.set_title('Actual vs. Predicted', fontsize = 14)
    ax_yy.set(xlabel = 'y_true')
    ax_yy.set(ylabel = 'y_pred')

    # в шестом - сравним распределение истинной и прогнозной целе
    sns.kdeplot(y, fill = True, ax = ax_ykde, label = 'y_true')
    sns.kdeplot(y_pred, fill = True, ax = ax_ykde, label = 'y_pre
    ax_ykde.set_title('Actual vs. Predicted Distribution', fontsi
    ax_ykde.set(xlabel = 'y_true and y_pred')
    ax_ykde.set(ylabel = 'Density')
    ax_ykde.legend(loc = 'upper right', prop = {'size': 12})

    plt.tight_layout()

    plt.show()
```

```
diagnostics(y, y_pred)
```





Разберем полученную информацию:

- в целом остатки модели распределены нормально с нулевым средним значением;
- явной гетероскедастичности нет, хотя мы видим, что дисперсия не всегда равномерна;
- присутствует умеренная отрицательная корреляция;
- график y_true vs. y_pred показывает, насколько сильно прогнозные значения отклоняются от фактических; в идеальной модели (без шума, т.е. без случайных колебаний) точки должны были бы находиться на диагонали, в более реалистичной модели нам бы хотелось видеть, что точки плотно сосредоточены вокруг диагонали;
- распределение прогнозных значений в целом повторяет распределение фактических.

Мультиколлинеарность

Отдельно проведем анализ на мультиколлинеарность. Напишем соответствующую функцию.

```
def vif(df, features):
    vif, tolerance = {}, {}

    # пройдемся по интересующим нас признакам
    for feature in features:
        # составим список остальных признаков, которые будем ис
        # для построения регрессии
        X = [f for f in features if f != feature]
        # поместим текущие признаки и таргет в X и y
        X, y = df[X], df[feature]
        # найдем коэффициент детерминации
        r2 = LinearRegression().fit(X, y).score(X, y)

        # посчитаем tolerance
        tolerance[feature] = 1 - r2
        # найдем VIF
        vif[feature] = 1 / (tolerance[feature])

    # выведем результат в виде датафрейма
    return pd.DataFrame({'VIF': vif, 'Tolerance': tolerance})
```

```
vif(df = X.drop('CHAS', axis = 1), features = X.drop('CHAS', axi
```

	VIF	Tolerance
CRIM	3.737294	0.267573
ZN	1.877035	0.532755
INDUS	3.538363	0.282617
NOX	5.169705	0.193435
RM	1.955011	0.511506
AGE	3.022372	0.330866
DIS	3.394467	0.294597
RAD	8.831025	0.113237
TAX	8.615462	0.116070
PTRATIO	1.463141	0.683461
B	1.262036	0.792370

LSTAT	3.704069	0.269973

Дополнительная обработка данных

Попробуем дополнительно улучшить некоторые из диагностических показателей.

VIF

Уберем признак с наибольшим VIF (RAD) и посмотрим, что получится.

```
vif(df = X, features = ['CRIM',
                        'ZN',
                        'INDUS',
                        'CHAS',
                        'NOX',
                        'RM',
                        'AGE',
                        'DIS',
                        'TAX',
                        'PTRATIO',
                        'B',
                        'LSTAT'])
```

	VIF	Tolerance
CRIM	2.943153	0.339772
ZN	1.865450	0.536064
INDUS	3.405180	0.293670
CHAS	1.045405	0.956567
NOX	5.019761	0.199213
RM	1.915038	0.522183
AGE	2.981066	0.335450
DIS	3.379355	0.295914

TAX	4.427160	0.225878
PTRATIO	1.450312	0.689507
B	1.250690	0.799559
LSTAT	3.717268	0.269015

Показатели пришли в норму. Окончательно удалим RAD.

```
boston.drop(columns = 'RAD', inplace = True)
```

Преобразование данных

Применим преобразование Йео-Джонсона.

```
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer()
boston = pd.DataFrame(pt.fit_transform(boston),
                      columns = boston.columns)
```

Отбор признаков

Посмотрим на линейную корреляцию Пирсона количественных признаков и целевой переменной.

```
boston_t.drop(columns = 'CHAS').corr().MEDV.to_frame().style.bac
```

	MEDV
CRIM	-0.594325
ZN	0.356857
INDUS	-0.598651
NOX	-0.571662
RM	0.621965

AGE	-0.548476
DIS	0.435669
TAX	-0.587527
PTRATIO	-0.552881
B	0.244196
LSTAT	-0.824408
MEDV	1.000000

Также рассчитаем точечно-бисериальную корреляцию.

```
pbc(boston_t.MEDV, boston_t.CHAS)
```

```
0.11362001957976946
```

Удалим признаки с наименьшей корреляцией, а именно
ZN, CHAS, DIS и B.

```
boston.drop(columns = ['ZN', 'CHAS', 'DIS', 'B'], inplace = True
```

Повторное моделирование и диагностика

Повторное моделирование

Выполним повторное моделирование.

```
X = boston_t.drop(columns = ['ZN', 'CHAS', 'DIS', 'B', 'MEDV'])
y = boston_t.MEDV

from sklearn.linear_model import LinearRegression

model = LinearRegression()
y_pred = model.fit(X, y).predict(X)
```

Оценка качества и диагностика

Оценим качество. Так как мы преобразовали целевую переменную, показатель RMSE не будет репрезентативен. Воспользуемся MAPE и R^2.

```
mape(y, y_pred)
```

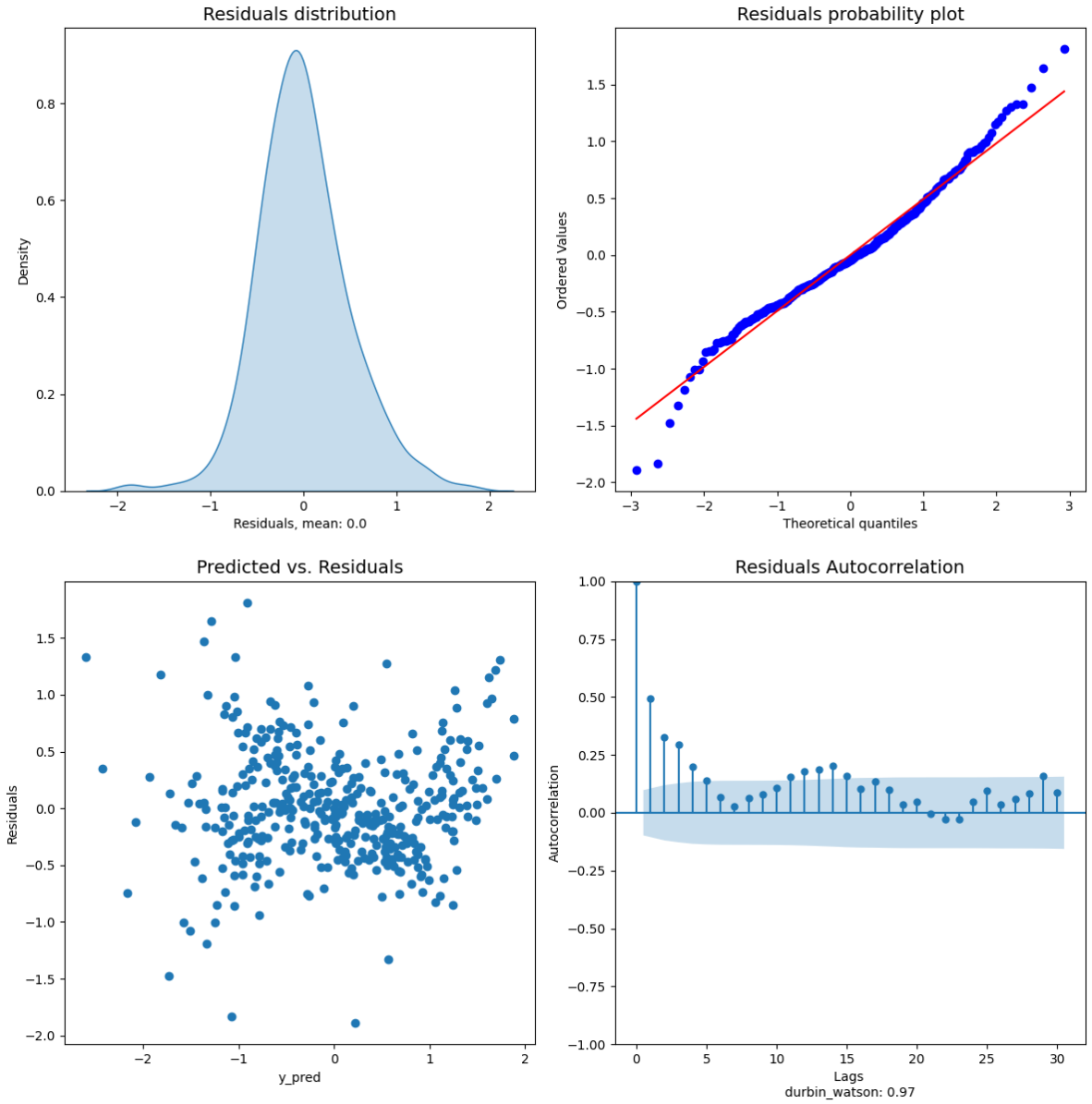
```
9.427283273312856
```

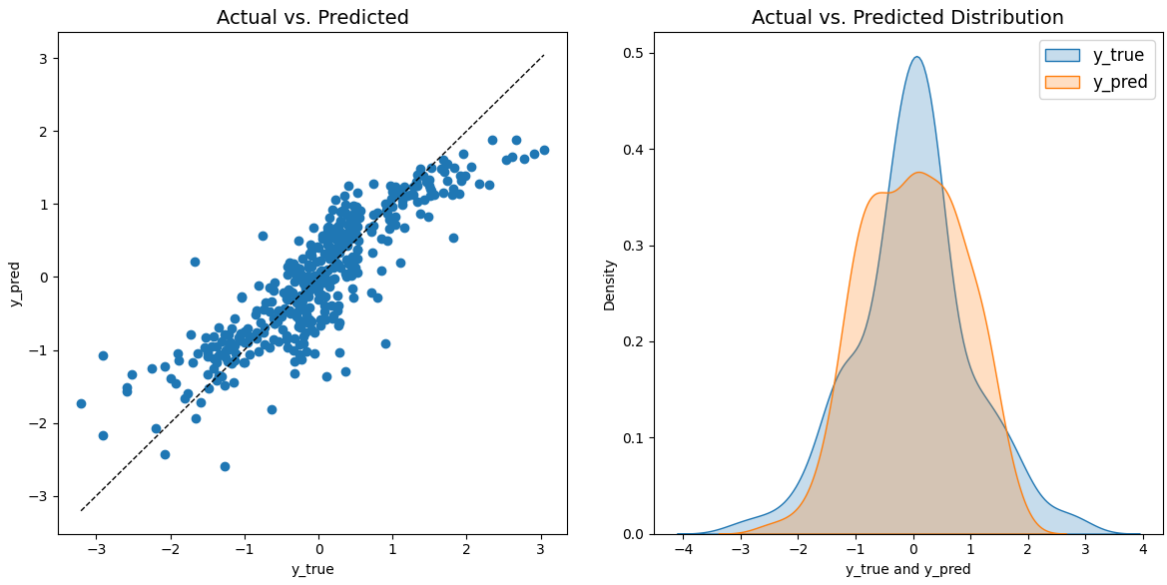
```
r_squared(X, y, y_pred)
```

```
(0.7546883770930229, 0.7546883770930229)
```

Отклонение прогнозного значения от истинного снизилось. R^2 немного уменьшился, чтобы бывает, когда мы пытаемся привести модель в соответствие допущениям. Проведем диагностику.

```
diagnostics(y, y_pred)
```





Распределение остатков немного улучшилось, при этом незначительно усилилась их отрицательная автокорреляция. Распределение целевой переменной стало менее островершинным.

Данные можно было бы продолжить анализировать и улучшать, однако в рамках текущего занятия перейдем к механике обучения модели.

Коэффициенты

Выведем коэффициенты для того, чтобы сравнивать их с результатами построенных с нуля моделей.

```
model.intercept_, model.coef_
```

```
(9.14334384717247e-16,
 array([-0.09989392,  0.03965441,  0.1069877 ,  0.23172172, -0.0
        -0.16878987, -0.18057055, -0.49319274]))
```

Обучение модели

Теперь, когда мы поближе познакомились с понятием регрессии, разобрали функции потерь и изучили допущения, при которых модель

может быть удачной аппроксимацией данных, пора перейти к непосредственному созданию алгоритмов.

Векторизация уравнения

Для удобства векторизуем приведенное ранее уравнение множественной линейной регрессии.

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, X = \begin{bmatrix} x_0 & x_1 & \dots & x_j \\ x_0 & x_1 & \dots & x_j \\ \vdots & \vdots & \vdots & \vdots \\ x_0 & x_1 & \dots & x_{n,j} \end{bmatrix}, \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

где n — количество наблюдений, а j — количество признаков.

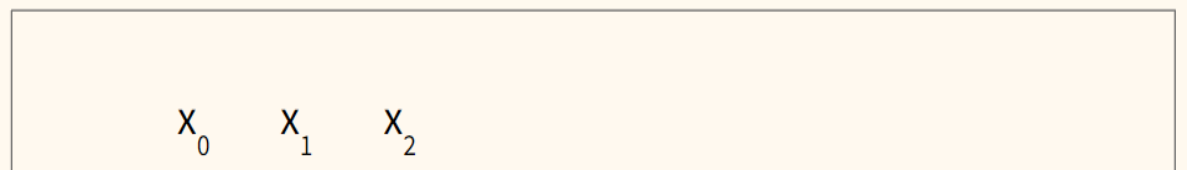
> Обратите внимание, что мы создали еще один столбец данных x_0, который будем умножать на сдвиг θ_0. Его мы заполним единицами.

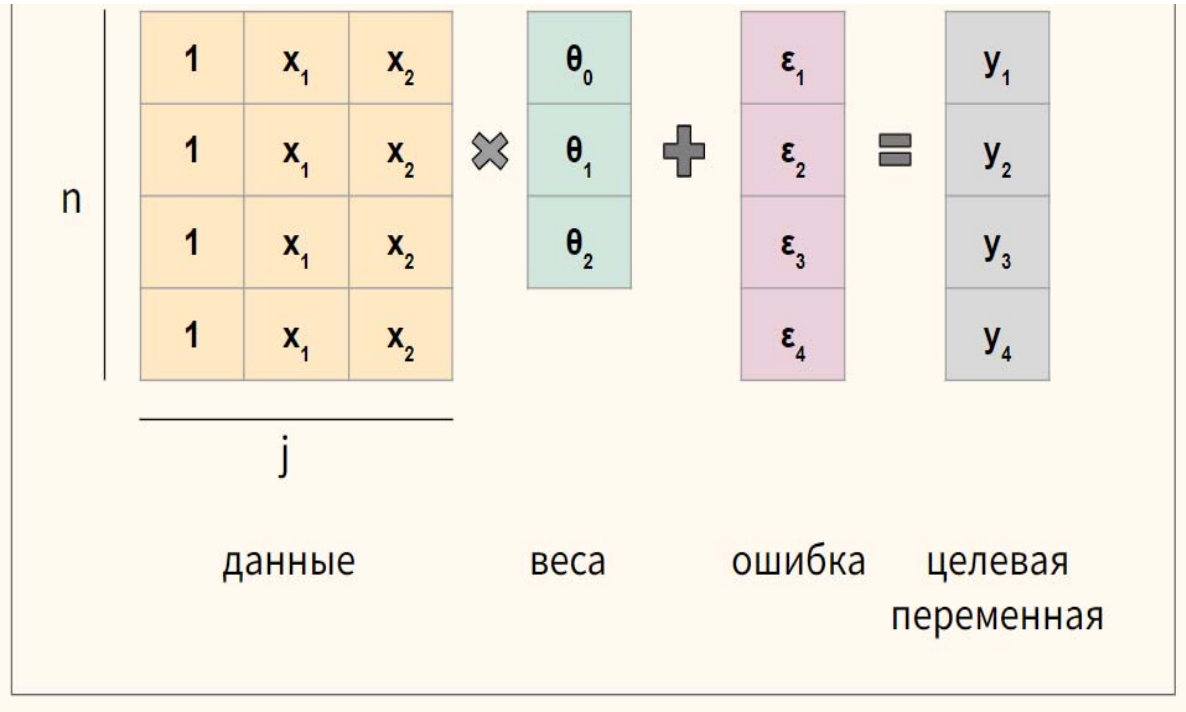
В результате такого несложного преобразования значение сдвига не изменится, но мы сможем записать уравнение через умножение матрицы на вектор.

$$y = X\theta + \varepsilon$$

Кроме того, как мы увидим ниже, так нам не придется искать отдельную производную для коэффициента θ_0.

Схематично для модели с четырьмя наблюдениями $(n = 4)$ и двумя признаками $(j = 2)$ получаются вот такие матрицы.





Функция потерь

Как мы уже говорили, чтобы подобрать оптимальные коэффициенты θ, нам нужен критерий или функция потерь. Логично измерять отклонение прогнозного значения от истинного.

$$\varepsilon = X\theta - y$$

При этом опять же просто складывать отклонения или ошибки мы не можем. Положительные и отрицательные значения будут взаимоудаляться. Для решения этой проблемы можно, например, использовать модуль числа, и это приводит нас к абсолютной ошибке или L1 loss.

Абсолютная ошибка, L1 loss

При усреднении на количество наблюдений мы получаем среднюю абсолютную ошибку (mean absolute error, MAE).

$$MAE = \frac{\sum |y - X\theta|}{n} = \frac{\sum |\varepsilon|}{n}$$

Приведем пример такой функции на Питоне.

```
def L1(y_true, y_pred):
  return np.sum(np.abs(y_true - y_pred)) / y_true.size
```

Помимо модуля ошибку можно возводить в квадрат.

Квадрат ошибки, L2 loss

В этом случай говорят про сумму квадратов ошибок (sum of squared errors, SSE) или сумму квадратов остатков (sum of squared residuals, SSR или residual sum of squares, RSS).

$$SSE = \sum (y - X\theta)^2$$

Как мы уже знаем, на практике вместо SSE часто используется MSE, или вернее half MSE для удобства нахождения производной.

$$MSE = \frac{1}{2n} \sum (y - \theta X)^2$$

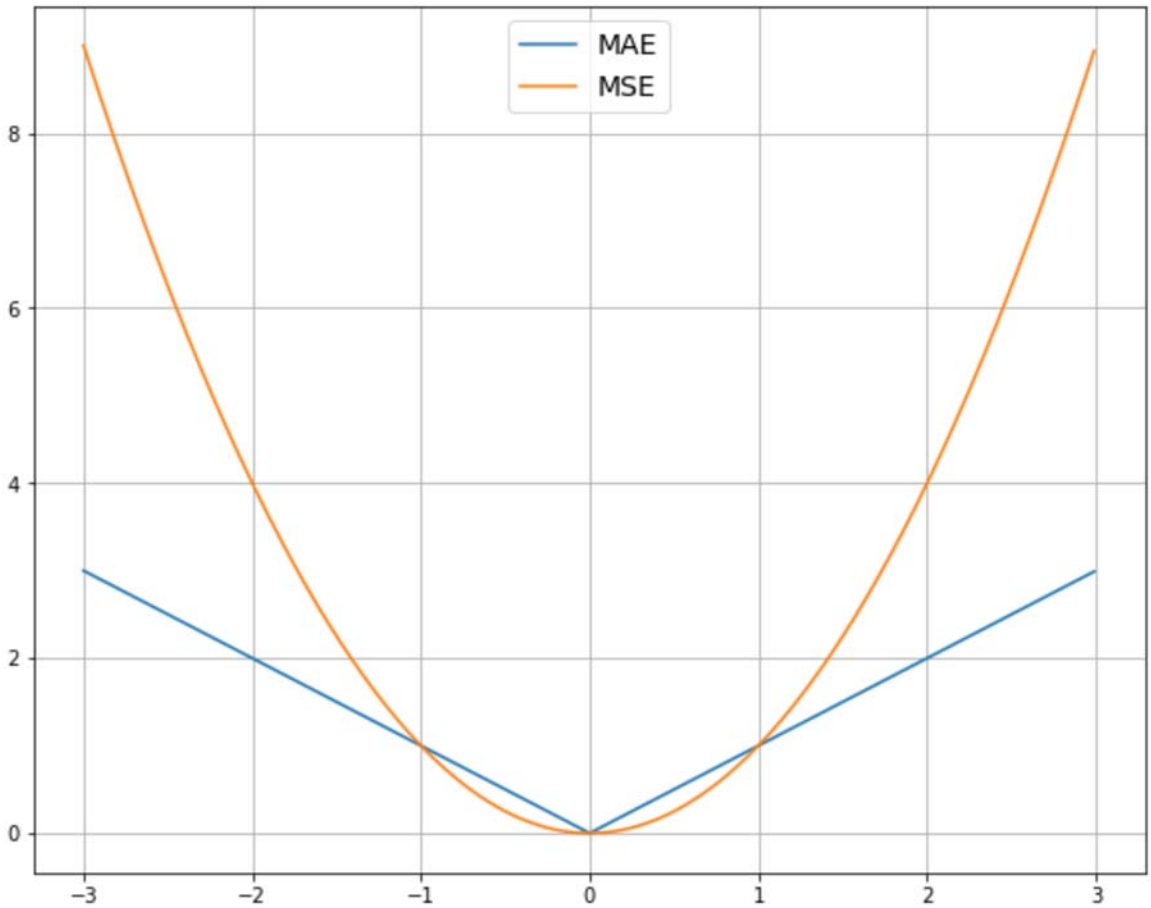
Ниже код на Питоне.

```
def L2(y_true, y_pred):
  return np.sum((y_true - y_pred) ** 2) / y_true.size
```

На практике у обеих функций есть сильные и слабые стороны. Рассмотрим L1 loss (MAE) и L2 loss (MSE) на графике.

```
# для построения графиков мы используем x вместо y_true, y_pred
# в качестве входящего значения
def mse(x):
  return x ** 2

def mae(x):
  return np.abs(x)

plt.figure(figsize = (10, 8))

x_vals = np.arange(-3, 3, 0.01)
```

```
12
13 plt.plot(x_vals, mae(x_vals), label = 'MAE')
14 plt.plot(x_vals, mse(x_vals), label = 'MSE')
15
16 plt.legend(loc = 'upper center', prop = {'size': 14})
17 plt.grid()
18 plt.show()
```



Очевидно, при отклонении от точки минимума из-за возведения в квадрат L2 значительно быстрее увеличивает ошибку, поэтому если в данных есть выбросы при суммированнии, они очень сильно влияют на этот показатель, хотя де-факто бóльшая часть значений такого уровня потерь не дали бы.

Функция L1 не дает такой большой ошибки на выбросах, однако ее сложно дифференцировать, в точке минимума ее производная не определена.

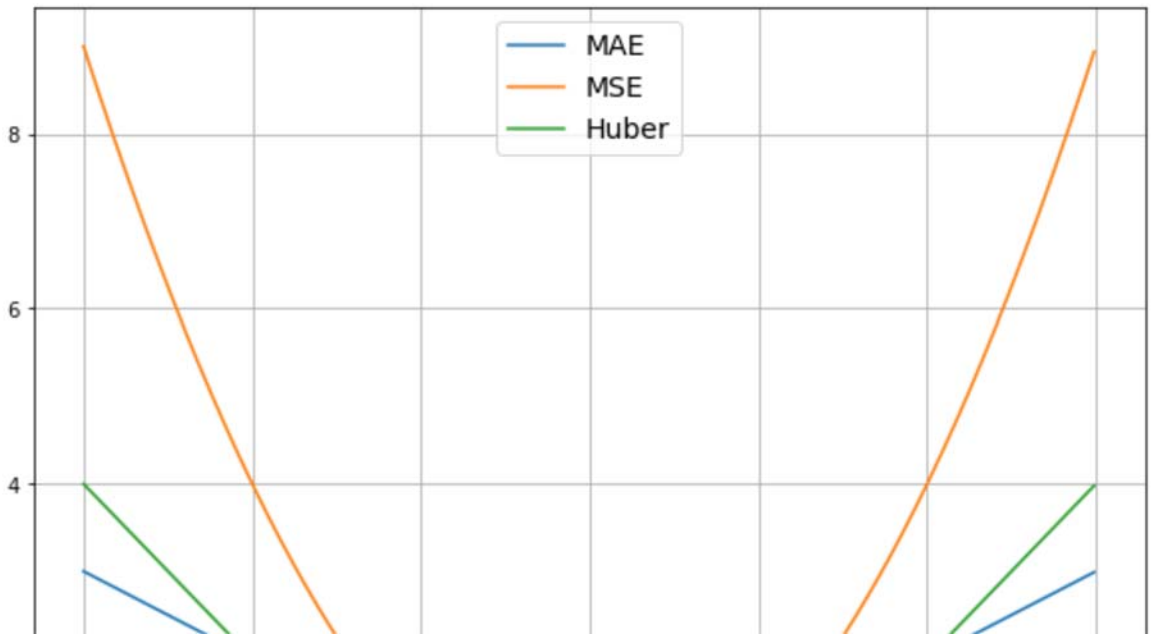
Функция Хьюбера

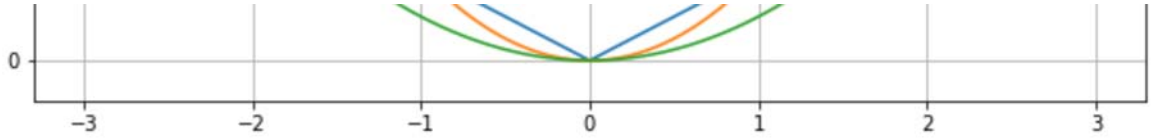
Рассмотрим функцию Хьюбера (Huber loss), которая объединяет сильные стороны вышеупомянутых функций и при этом лишена их недостатков. Посмотрим на формулу.

$$L_\delta = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{if } |y-\hat{y}| < \delta \\ \delta(|y-\hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

Выведем ее на графике.

```
plt.figure(figsize = (10, 8))

def huber(x, delta = 1.):
  huber_mse = 0.5 * np.square(x)
  huber_mae = delta * (np.abs(x) - 0.5 * delta)
  return np.where(np.abs(x) <= delta, huber_mse, huber_mae)

x_vals = np.arange(-3, 3, 0.01)

plt.plot(x_vals, mae(x_vals), label = 'MAE')
plt.plot(x_vals, mse(x_vals), label = 'MSE')
plt.plot(x_vals, huber(x_vals, delta = 2), label = 'Huber')

plt.legend(loc = 'upper center', prop = {'size': 14})
plt.grid()
plt.show()
```





Также приведем код этой функции.

```
def huber(y_pred, y_true, delta = 1.0):
  # пропишем обе части функции потерь
  huber_mse = 0.5 * (y_true - y_pred) ** 2
  huber_mae = delta * (np.abs(y_true - y_pred) - 0.5 * delta)
  # выберем одну из них в зависимости от дельта
  return np.where(np.abs(y_true - y_pred) <= delta, huber_mse, h
```

На сегодняшнем занятии мы, как и раньше, в качестве функции потерь используем MSE.

Метод наименьших квадратов

Нормальные уравнения

Для множественной линейной регрессии коэффициенты находятся по следующей формуле

$$\theta = (X^T X)^{-1} X^T y$$

Разберем эту формулу подробнее. Сумму квадратов остатков (SSE) можно переписать как произведение вектора $\hat{\varepsilon}$ на самого себя, то есть $SSE = \varepsilon^T \varepsilon$. Помня, что $\varepsilon = y - X\theta$ получаем (не забывая транспонировать)

$$(y - X\theta)^T (y - X\theta)$$

$$y^T y - y^T (X\theta) - (X\theta)^T y + (X\theta)^T (X\theta)$$

Заметим, что $A^T B = B^T A$, тогда

$$y^T y - (X\theta)^T y - (X\theta)^T y + (X\theta)^T (X\theta)$$

$$y^Ty - 2(X\theta)^Ty + (X\theta)^T(X\theta)$$

Вспомним, что $(AB)^T = A^TB^T$, тогда

$$y^Ty - 2\theta^TX^Ty + \theta^TX^TX\theta$$

Теперь нужно найти градиент этой функции

$$\nabla_{J(\theta)}\left(y^Ty - 2\theta^TX^Ty + \theta^TX^TX\theta\right)$$

После дифференцирования относительно θ мы получаем следующий результат

$$\nabla_{J(\theta)} = -2X^Ty + 2X^TX\theta$$

Как мы помним, оптимум функции находится там, где производная равна нулю.

$$-2X^Ty + 2X^TX\theta = 0$$

$$-X^Ty + X^TX\theta = 0$$

$$X^TX\theta = X^Ty$$

Выражение выше называется **нормальным уравнением** (normal equation). Решив его для θ, мы найдем аналитическое решение минимизации суммы квадратов отклонений.

$$\theta = (X^TX)^{-1}X^Ty$$

> По теореме Гаусса-Маркова, оценка через МНК является наиболее оптимальной (обладающей наименьшей дисперсией) среди всех методов построения модели.

Код на Питоне

Перейдем к созданию класса линейной регрессии наподобие LinearRegression библиотеки sklearn. Вначале напишем функцию гипотезы (т.е. функцию самой модели), снабдив ее еще одной функцией, которая добавляет столбец из единиц к признакам.

$$h_\theta(x) = \theta X$$

```
def add_ones(x):
  # важно! метод .insert() изменяет исходный датафрейм
  return x.insert(0,'x0', np.ones(x.shape[0]))
```

```
def h(x, theta):
  x = x.copy()
  add_ones(x)
  return np.dot(x, theta)
```

Перейдем к функции, отвечающей за обучение модели.

$$\theta = (X^T X)^{-1} X^T y$$

```
# строчную x используем внутри функций и методов класса
# заглавную X вне функций и методов
def fit(x, y):
  x = x.copy()
  add_ones(x)
  xT = x.transpose()

  inversed = np.linalg.inv(np.dot(xT, x))
  thetas = inversed.dot(xT).dot(y)

  return thetas
```

Обучим модель и выведем коэффициенты.

```
thetas = fit(X, y)
thetas[0], thetas[1:]
```

```
(8.959846753420209e-16,
 array([-0.09989392,  0.03965441,  0.1069877 ,  0.23172172, -0.0
        -0.16878987, -0.18057055, -0.49319274]))
```

Примечание. Замечу, что не все матрицы обратимы. В этом случае можно найти *псевдообратную* матрицу (pseudoinverse). Для этого в Numpy есть **функция np.linalg.pinv()**.

Сделаем прогноз.

```
y_pred = h(X, thetas)
y_pred[:5]
```

```
array([1.24414666, 0.55999778, 1.48103299, 1.49481605, 1.2134278
```

Создание класса

Объединим код в класс.

```
class Ols():
  def __init__(self):
    self.thetas = None

  def add_ones(self, x):
    return x.insert(0,'x0', np.ones(x.shape[0]))

  def fit(self, x, y):
    x = x.copy()
    self.add_ones(x)
    xT = x.T

    inversed = np.linalg.inv(np.dot(xT, x))
    self.thetas = inversed.dot(xT).dot(y)

  def predict(self, x):
    x = x.copy()
    self.add_ones(x)
    return np.dot(x, self.thetas)
```

Создадим объект класса и обучим модель.

```
model = Ols()

model.fit(X, y)
```

Выведем коэффициенты.

```
model.thetas[0], model.thetas[1:]
```

```
(8.959846753420209e-16,
 array([-0.09989392,  0.03965441,  0.1069877 ,  0.23172172, -0.0
```

```
        -0.16878987, -0.18057055, -0.49319274]))
```

Сделаем прогноз.

```
y_pred = model.predict(X)
y_pred[:5]
```

```
array([1.24414666, 0.55999778, 1.48103299, 1.49481605, 1.2134278
```

Оценка качества

Оценим качество через MAPE и R^2.

```
mape(y, y_pred)
```

```
9.427283273312836
```

```
r_squared(X, y, y_pred)
```

```
(0.7546883770930229, 0.7546883770930229)
```

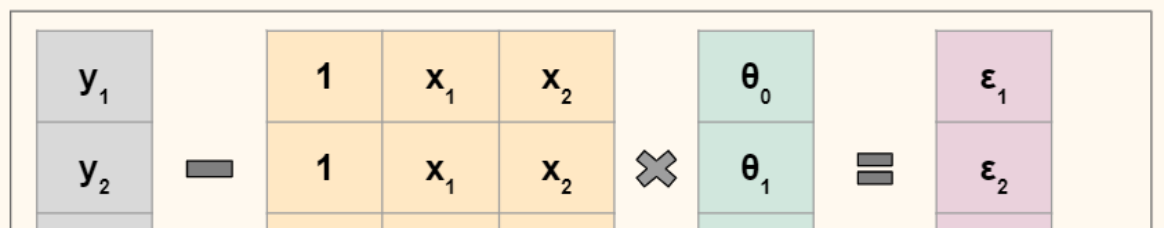
Мы видим, что результаты аналогичны.

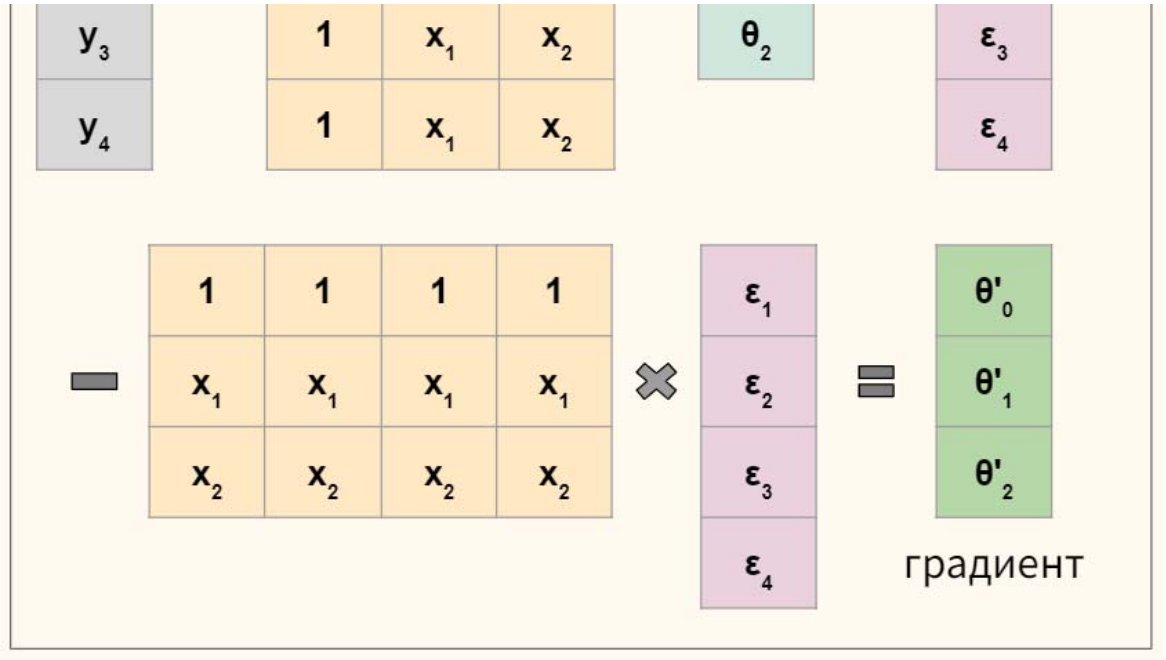
Метод градиентного спуска

В целом с этим методом мы уже хорошо знакомы. В качестве упражнения давайте реализуем этот алгоритм на Питоне для многомерных данных.

Нахождение градиента

Покажем расчет градиента на схеме.





В данном случае мы берем датасет из четырех наблюдений и двух признаков x_1 и x_2, и соответственно используем три коэффициента θ_0, θ_1 и θ_2.

Пошаговое построение модели

Начнем с функции гипотезы.

$$h_\theta(x) = X\theta$$

```
def h(x, thetas):
  return np.dot(x, thetas)
```

Объявим функцию потерь.

$$J(\theta_j) = \frac{1}{2n} \sum (y - X\theta)^2$$

```
def objective(x, y, thetas, n):
  return np.sum((y - h(x, thetas)) ** 2) / (2 * n)
```

Найдем частную производную для каждого j признака.

$$\frac{\partial J}{\partial \theta_j} = -X(y - X\theta_j) \times \frac{1}{n} = X(X\theta_j - y) \times \frac{1}{n}$$

Градиент (совокупность этих производных) будет иметь вид

$$\nabla_\theta J = X^T \cdot (X\theta - y) \times \frac{1}{n}$$

Примечание. В данном случае оператор $\cdot$ означает скалярное произведение, а $\times$ — произведение Адамара.

```
def gradient(x, y, thetas, n):
  return np.dot(x.T, (h(x, thetas) - y)) * (1/n)
```

Напишем функцию для обучения модели.

$$\theta := \theta - \alpha \nabla_\theta J$$

Символ $:=$ означает, что левая часть равенства определяется правой. По сути, с каждой итерацией мы обновляем веса, умножая коэффициент скорости обучения на градиент.

```
def fit(x, y, iter = 20000, learning_rate = 0.05):
  x, y = x.copy(), y.copy()

  # функцию add_ones() мы объявили выше
  add_ones(x)

  thetas, n = np.zeros(x.shape[1]), x.shape[0]

  loss_history = []

  for i in range(iter):
    loss_history.append(objective(x, y, thetas, n))
    grad = gradient(x, y, thetas, n)
    thetas -= learning_rate * grad

  return thetas, loss_history
```

Обучим модель, выведем коэффициенты и достигнутый (минимальный) уровень ошибки.

```
thetas, loss_history = fit(X, y, iter = 50000, learning_rate = 0
```

```
thetas[0], thetas[1:], loss_history[-1]
```

```
(9.097200604764374e-16,
```

```
array([-0.09989392,  0.03965441,  0.1069877 ,  0.23172172, -0.0
       -0.16878987, -0.18057055, -0.49319274]),
0.12265581145348856)
```

Полученный результат очень близок к тому, что было найдено методом наименьших квадратов.

Прогноз

Сделаем прогноз.

```
def predict(x, thetas):
  x = x.copy()
  add_ones(x)
  return np.dot(x, thetas)
```

```
y_pred = predict(X, thetas)
y_pred[:5]
```

```
array([1.24414666, 0.55999778, 1.48103299, 1.49481605, 1.2134278
```

Создание класса

Объединим написанные функции в класс.

```
class Gd():
  def __init__(self):
    self.thetas = None
    self.loss_history = []

  def add_ones(self, x):
    return x.insert(0,'x0', np.ones(x.shape[0]))

  def objective(self, x, y, thetas, n):
    return np.sum((y - self.h(x, thetas)) ** 2) / (2 * n)

  def h(self, x, thetas):
    return np.dot(x, thetas)

  def gradient(self, x, y, thetas, n):
    return np.dot(-x.T, (y - self.h(x, thetas))) / n
```

```

  def fit(self, x, y, iter = 20000, learning_rate = 0.05):
    x, y = x.copy(), y.copy()
    self.add_ones(x)

    thetas, n = np.zeros(x.shape[1]), x.shape[0]

    # объявляем переменную loss_history (отличается от self.los
    loss_history = []

    for i in range(iter):
      loss_history.append(self.objective(x, y, thetas, n))
      grad = self.gradient(x, y, thetas, n)
      thetas -= learning_rate * grad

    # записываем обратно во внутренние атрибуты, чтобы передать
    self.thetas = thetas
    self.loss_history = loss_history

  def predict(self, x):
    x = x.copy()
    self.add_ones(x)
    return np.dot(x, self.thetas)
```

Создадим объект класса, обучим модель, выведем коэффициенты и сделаем прогноз.

```
model = Gd()

model.fit(X, y, iter = 50000, learning_rate = 0.05)

model.thetas[0], model.thetas[1:], model.loss_history[-1]
```

```
(9.097200604764374e-16,
 array([-0.09989392,  0.03965441,  0.1069877 ,  0.23172172, -0.0
        -0.16878987, -0.18057055, -0.49319274]),
 0.12265581145348856)
```

```
y_pred = model.predict(X)
y_pred[:5]
```

```
array([1.24414666, 0.55999778, 1.48103299, 1.49481605, 1.2134278
```

Оценка качества

```
mape(y, y_pred)
```

```
9.427283273312812
```

```
r_squared(X, y, y_pred)
```

```
(0.7546883770930229, 0.7546883770930229)
```

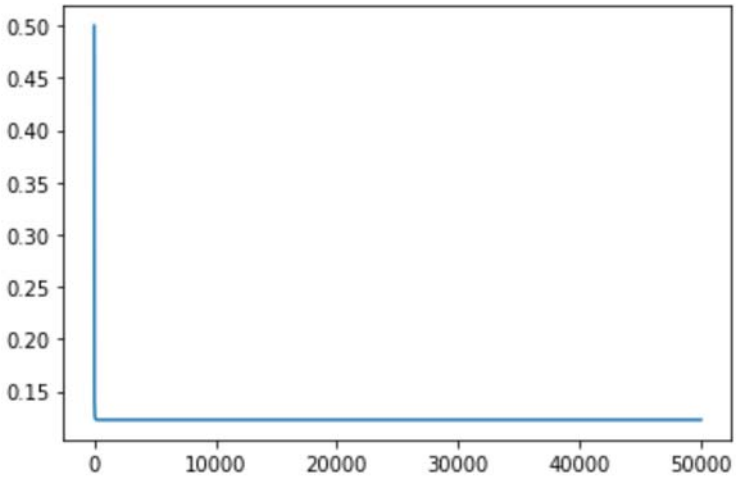
Диагностика алгоритма

Работу алгоритма можно проверить с помощью кривой обучения (learning curve), на которой мы должны увидеть, что:

- ошибка постоянно снижается;
- алгоритм остановится после истечения заданного количества итераций.

При этом можно также задать пороговое значение ошибки, после которого обновление весов прекратится. Построим кривую обучения.

```
plt.plot(loss_history);
```




```
plt.plot(loss_history[:100]);
```

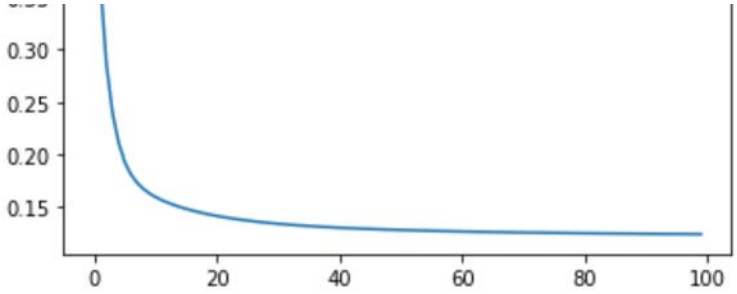


График также позволяет выбрать адекватный коэффициент скорости обучения.

Подведем итог

Сегодня мы подробно рассмотрели модель множественной линейной регрессии. В частности, мы поговорили про построение гипотезы, основные функции потерь, допущения модели линейной регрессии, метрики качества и диагностику модели.

Кроме того, мы узнали, как изнутри устроены метод наименьших квадратов и метод градиентного спуска, и построили соответствующие модели на Питоне.